

ViT, Swin Transformer, CvT

3조 서영, 김기연, 이윤지

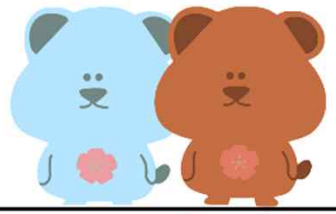
EWHA EURON

목차

#01 ViT

#02 ViT 모델 구조

#03 CvT



EWHA EURON

#1 전체 개요

#1 ViT

이미지 인식을 위한 딥러닝 모델

이미지를 자연어 처리처럼 분류해보려는 시도에서 등장

CNN: 합성곱 layer를 이용

→ ViT: 트랜스포머에서 사용되는 self-attention을 적용, self attention -> 전체 이미지를 한 번에 처리

#1 전체 개요

#2 ViT, Swin Transformer, CvT

ViT의 한계: 이미지 패치가 왼 → 오, 위 → 아래 방향으로 순차적으로 입력되어 2차원 구조의 이미지 특성을 온전히 반영하지 못한다.

➔ Swin Transformer, CvT 모델이 등장

Swin Transformer: 로컬위도(local window)를 통해 각 계층의 어텐션 되는 패치의 크기와 개수를 다양하게 구성하여 이미지의 특징을 학습하는 방법

CvT: 기존 합성곱 연산과정을 ViT에 적용, 저수준 특징과 고수준 특징을 계층적으로 반영, 계산복잡도 감소

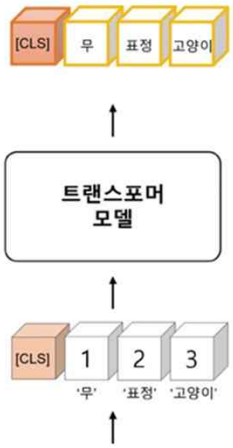
EWHA EURON

#2 ViT 모델

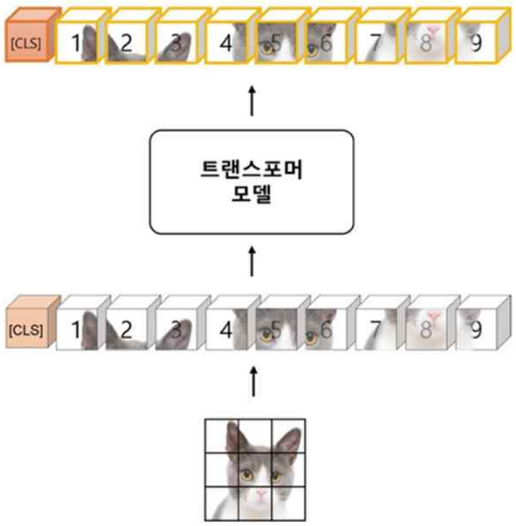
#1 ViT vs BERT

BERT 모델

- 자연어 처리 모델
- 문장보다 작은 단위의 토큰들이 순차적으로 입력됨



< BERT 모델 구조 >



<ViT 모델 구조>

ViT 모델

- 이미지를 격자로 나눈 이미지 패치로 나누고 순차적으로 입력됨
- 이미지 패치 = 왼 → 오, 위 → 아래 방향으로 표현된 시퀀셜(sequential) 배열

#2 ViT 모델

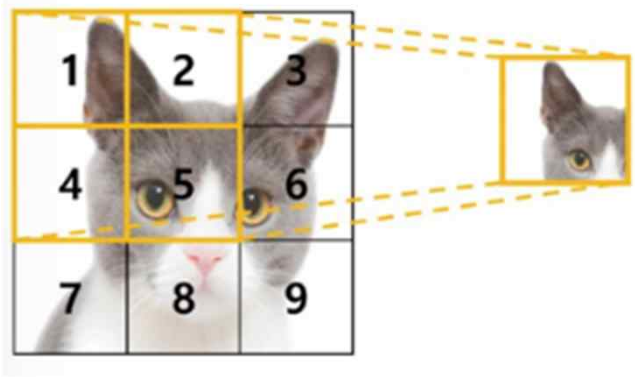
#2 CNN vs ViT

공통점: CNN의 합성곱 과 ViT의 트랜스포머 둘다 이미지를 잘 표현하는 임베딩을 만들고자 한다는 점에서는 같다.

그러나 그 과정이 다르다.

CNN 합성곱 신경망

- 이미지 패치 중 일부만 선택하여 학습 → 이미지 전체의 특징을 추출
- Ex) 고양이 이미지에서 왼쪽 눈의 특징을 추출하자! → 이미지에서 해당 부분만 학습
- 좁은 수용 영역(receptive field)를 가져서,
  전체 이미지 정보를 표현하는데 여러 계층이 필요
- 이미지의 공간적인 위치 정보를 고려 → 이미지 변환에 강함



#2 ViT 모델

#2 CNN vs ViT

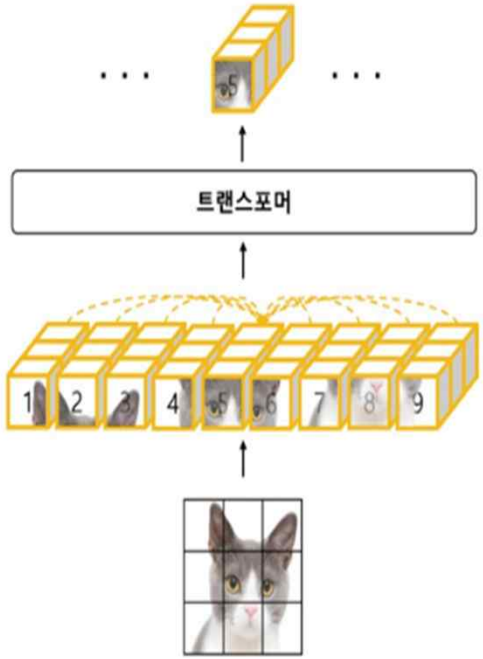
공통점: CNN의 합성곱 과 ViT의 트랜스포머 둘다 이미지를 잘 표현하는 임베딩을 만들고자 한다는 점에서는 같다.

그러나 그 과정이 다르다.

ViT 트랜스포머

- 이미지를 작은 패치들로 나눔 → 각 패치 간의 상관관계를 학습
- Self attention 방법을 사용 → 모든 이미지 패치가 서로에게 주는 영향을 고려
- 어텐션 거리를 계산 → 1개의 ViT 레이어로 전체 이미지 정보를 표현
- CNN 보다 더 작은 모델로도 높은 성능을 보임
- 입력 이미지의 크기가 고정되어있어서,

  → 크기가 다른 이미지는 이미지 크기를 맞추는 전처리 필요
- 패치 간의 상대적인 위치 정보만을 고려 → 이미지 변환에 취약



#2 ViT 모델

#3 귀납적 편향 Inductive Bias

의미

- 데이터를 학습하기 전부터 가지고 있는 기본적인 가정
- 이미지의 특징에 대한 사전 지식이 모델 구조에 거의 반영되어 있지 않다

예시) 고양이 사진 분류

- 가정1) 가까운 픽셀 끼리는 서로 연관이 있을 것이다.
- 가정2) 고양이가 왼쪽에 있든, 오른쪽에 있든 고양이라는 사실은 변하지 않는다.
- 이러한 가정을 모델 구조에 반영하는 것이 “귀납적 편향”





EWHA EURON

#2 ViT 모델

#3 귀납적 편향 Inductive Bias

CNN 합성곱 신경망: 지역적 편향, 귀납적 편향

- 이미지를 작은 커널을 통해 특징 추출
- 이미지는 가까운 픽셀끼리 관련이 깊을 거라는 가정을 가지고 학습하는 모델

RNN 순환 신경망: 순차적 편향

- 이전 시간의 상태를 기억, 현재 입력과 결합 → 다음 상태 예측
- 시계열 데이터에서 좋은 성능
- 다른 다양한 관계를 표현하기가 어려움 → 낮은 귀납적 편향

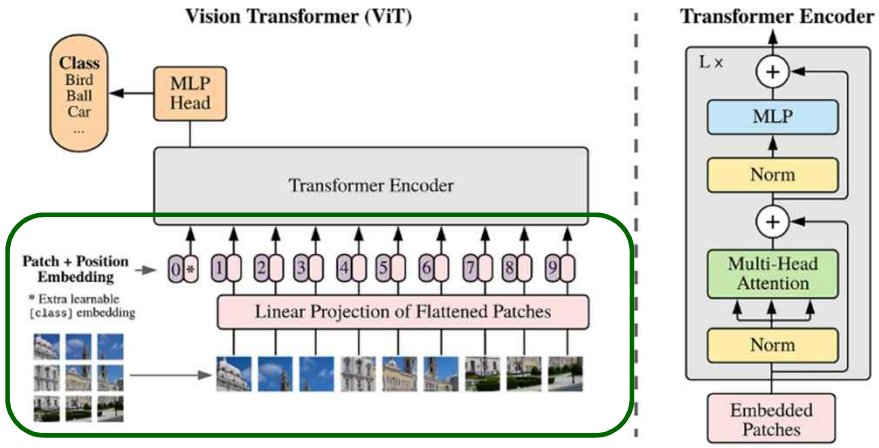
ViT: 귀납적 편향 거의 없음

- ViT의 self-attention은 가까운 패치가 반드시 더 중요하다고 가정하지 않음
- 멀리 떨어진 패치 사이의 관계도 직접 학습이 가능
- 입력데이터의 다양한 쿼리, 키, 값의 임베딩 형태로 일반화된 관계를 학습

EWHA EURON

#3 ViT 모델 구조

#1 패치 임베딩 Patch Embedding



1. 이미지 크기를 맞추는 전처리
2. 전체 이미지를 패치 크기로 분할
3. 패치 임베딩: 각 패치를 펼친 후, 선형 변환하여 벡터로 만듦
4. 배열 가장 왼쪽에 CLS, 분류토큰을 추가
5. 위치 임베딩(position embedding)을 통해 패치의 위치 정보를 임베딩 벡터로 변환
6. 임베딩 벡터를 기존 이미지 패치 벡터들과 더함
7. 계층 정규화

EWHA EURON

#3 ViT 모델 구조

#1 패치 임베딩 Patch Embedding

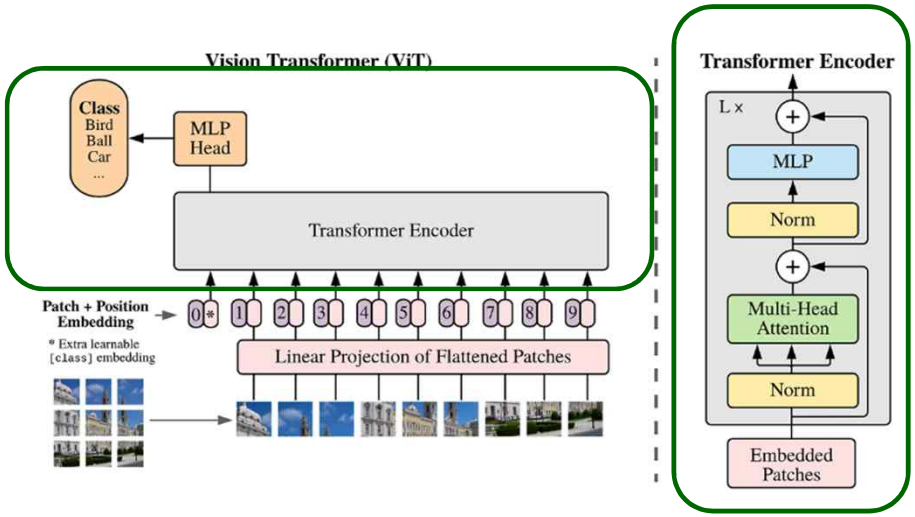
- CLS, 분류토큰: 전체 이미지를 대표하는 벡터, 특정 문제를 예측하는 데 사용됨
- 패치 크기로 분할할 때, 커널크기와 stride는 하이퍼파라미터로 정의
- 만들어지는 패치크기

→ patch_size=((image_size – kernel_size) / (stride)) +1

EWHA EURON

#3 ViT 모델 구조

#2 인코더 계층



- 패치 임베딩을 통해 만들어진 벡터를 인코더 계층으로 넣음
- 트랜스포머의 학습과정과 동일
- N개의 인코더 레이어를 반복 적용
- 마지막 레이어에서 CLS(분류토큰 벡터)를 추출
- Ex) 고양이인지 아닌지를 구분하는 문제에서, ViT 모델은 CLS 벡터만 사용해서 분류 문제를 풀어냄

#4 모델 실습

#0 실습 데이터

- 허깅페이스
- FashionMNIST : 의류 이미지, 10개 클래스의 훈련 데이터세트, 10,000개의 테스트 데이터세트

EWHA EURON

#4 모델 실습

#1 FashionMNIST 다운로드

```
from itertools import chain
from collections import defaultdict
from torch.utils.data import Subset
from torchvision import datasets

def subset_sampler(dataset, classes, max_len):
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.train_labels):
        target_idx[int(label)].append(idx)

    indices = list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)
```

subset_sampler: 데이터를 클래스별로 분류하고, 각 클래스에서 일부 데이터만 골라냄

데이터세트, 클래스 목록, 클래스별 최대 샘플링 개수를 전달

EWHA EURON

#4 모델 실습

#1 FashionMNIST 다운로드

```
train_dataset = datasets.FashionMNIST(root="../datasets", download=True, train=True)
test_dataset = datasets.FashionMNIST(root="../datasets", download=True, train=False)

classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

print(classes)
print(class_to_idx)

subset_train_dataset = subset_sampler(
    dataset=train_dataset, classes=train_dataset.classes, max_len=1000
)
subset_test_dataset = subset_sampler(
    dataset=test_dataset, classes=test_dataset.classes, max_len=100
)
```

FashionMNIST 데이터세트에 포함된 클래스--> classes 속성

클래스 ID, 클래스가 매핑된 값 → class_to_idx 속성

EWHA EURON

#4 모델 실습

#2 이미지 전처리

```
import torch
from torchvision import transforms
from transformers import AutoImageProcessor

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="google/vit-base-patch16-224-in21k"
)

transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize(
            size=(
                image_processor.size["height"],
                image_processor.size["width"]
            )
        ),
        transforms.Lambda(
            lambda x: torch.cat([x, x, x], 0)
        ),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
```

AutoImageProcessor

- 사전학습된 ViT모델로 전처리 진행
- 자르기(crop), 크기 조정(resize) 등의 전처리 가능

- transforms.ToTensor(): 모델 학습 시 PIL.Image 형식이 아닌, Tensor 형식을 사용
- transforms.Resize : 크기 조정
- transforms.Lambda : 단일 채널을 복제, 다중 채널 이미지로 변환
- Transforms.Normalize: 정규화

EWHA EURON

#4 모델 실습

#2 데이터로더

```
from torch.utils.data import DataLoader

def collator(data, transform):
    images, labels = zip(*data)
    pixel_values = torch.stack([transform(image) for image in images])
    labels = torch.tensor([label for label in labels])
    return {"pixel_values": pixel_values, "labels": labels}

train_dataloader = DataLoader(
    subset_train_dataset,
    batch_size=32,
    shuffle=True,
    collate_fn=lambda x: collator(x, transform),
    drop_last=True
)

valid_dataloader = DataLoader(
    subset_test_dataset,
    batch_size=4,
    shuffle=True,
    collate_fn=lambda x: collator(x, transform),
    drop_last=True
)
```

Collator 함수

- 역할: 여러 이미지와 정답 라벨을 ViT 모델에 입력할 수 있는 형태로 변환
- Images, labels=zip(*data): DataLoader가 전달한 여러 샘플을 이미지와 라벨로 분리
- Transform(image) : 각 이미지를 ViT 입력에 적합한 텐서로 전처리
  → torch.stack(): 전처리된 이미지 텐서들을 하나의 배치 텐서로 쌓음
- torch.tensor([label for label in labels]) : 각 이미지의 정답 라벨을 텐서로 변환
- 반환: 이미지 배치와 라벨 배치를 딕셔너리로 묶어서 반환

Train_dataloader

- 역할: 학습 데이터를 32개씩 묶어 모델에 공급하는 DataLoader를 생성
- Subset_train_dataset: 학습에 사용할 데이터세트
- Batch_size: 32
- Shuffle=True: 데이터의 순서를 무작위로 섞음
- Collate_fn: 샘플들을 collator() 에 전달해 배치로 변환
- Drop_last=True: 마지막 배치에 32개보다 적은 데이터가 남으면 버림

EWHA EURON

#4 모델 실습

#2 데이터로더

```
from torch.utils.data import DataLoader

def collator(data, transform):
    images, labels = zip(*data)
    pixel_values = torch.stack([transform(image) for image in images])
    labels = torch.tensor([label for label in labels])
    return {"pixel_values": pixel_values, "labels": labels}

train_dataloader = DataLoader(
    subset_train_dataset,
    batch_size=32,
    shuffle=True,
    collate_fn=lambda x: collator(x, transform),
    drop_last=True
)
valid_dataloader = DataLoader(
    subset_test_dataset,
    batch_size=4,
    shuffle=True,
    collate_fn=lambda x: collator(x, transform),
    drop_last=True
)
```

Collator 함수

- 역할: 여러 이미지와 정답 라벨을 ViT 모델에 입력할 수 있는 형태로 변환
- Images, labels=zip(*data): DataLoader가 전달한 여러 샘플을 이미지와 라벨로 분리
- Transform(image) : 각 이미지를 ViT 입력에 적합한 텐서로 전처리
  → torch.stack(): 전처리된 이미지 텐서들을 하나의 배치 텐서로 쌓음
- torch.tensor([label for label in labels]) : 각 이미지의 정답 라벨을 텐서로 변환
- 반환: 이미지 배치와 라벨 배치를 딕셔너리로 묶어서 반환

Valid_dataloader

- 역할: 검증용 데이터를 4개씩 묶어 모델에 공급하는 DataLoader를 생성
- Subset_train_dataset: 학습에 사용할 데이터세트
- Batch_size: 4
- Shuffle=True: 데이터의 순서를 무작위로 섞음
- Collate_루: 샘플들을 collator() 에 전달해 배치로 변환
- Drop_last=True: 마지막 배치에 32개보다 적은 데이터가 남으면 버림

EWHA EURON

#4 모델 실습

#3 사전 학습된 ViT 모델

```
from transformers import ViTForImageClassification

model = ViTForImageClassification.from_pretrained(
    pretrained_model_name_or_path="google/vit-base-patch16-224-in21k",
    num_labels=len(classes),
    id2label={idx: label for label, idx in class_to_idx.items()},
    label2id=class_to_idx,
    ignore_mismatched_sizes=True
)

print(model.classifier)
```

```
Linear(in_features=768, out_features=10, bias=True)
```

ViTForImageCloassification

- 허깅페이스 라이브러리에서 사전 학습된 모델을 불러옴
- 21841개의 분류 카테고리로 학습된 모델 → 미세 조정이 필요
- 레이블개수(num_labels), ID/레이블(id2label), 레이블/ID(label2id)

출력결과

→ 분류기(classifier)의 출력데이터 차원 크기(out_features) 가 10개로 나옴

EWHA EURON

#4 모델 실습

#4 패치 임베딩

```
print(model.vit.embeddings)

batch = next(iter(train_dataloader))
print("image shape :", batch["pixel_values"].shape)
print("patch embeddings shape :",
    model.vit.embeddings.patch_embeddings(batch["pixel_values"]).shape
)
print("[CLS] + patch embeddings shape :",
    model.vit.embeddings(batch["pixel_values"]).shape
)
```

```
ViTEmbeddings(
  (patch_embeddings): ViTPatchEmbeddings(
    (projection): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
  )
  (dropout): Dropout(p=0.0, inplace=False)
)
image shape : torch.Size([32, 3, 224, 224])
patch embeddings shape : torch.Size([32, 196, 768])
[CLS] + patch embeddings shape : torch.Size([32, 197, 768])
```

ViT 모델에서 이미지가 패치 임베딩으로 변환되고
이후 CLS 토큰과 위치 임베딩이 추가되는 과정을 확인하는 코드

Batch=next(iter(train_dataloader)) :

- iter() : DataLoader를 반복가능한 iterator로 변환
- Next() : iterator 에서 첫번째 미니 배치를 가져옴
- 앞에서 배치 사이즈를 32개로 설정했으므로 이미지 32개를 가져옴

출력결과

1) 패치 임베딩의 합성곱 구조
2) 원본 이미지 : (32,3,224,224) : 배치크기, 이미지 채널수(RGB), 이미지 높이, 너비
3) 패치 임베딩: (32,96,768): 배치 크기, 이미지 한장을 16*16으로 분할 시 나오는 패치 개수, 임베딩 벡터의 차원 수
4)최종 임베딩: (32,197,768): 배치크기, 패치 196개 + CLS 1개=197, 임베딩 벡터의 차원 수

EWHA EURON

#4 모델 실습

#5 하이퍼파라미터 설정

```
args = TrainingArguments(
    output_dir="../models/ViT-FashionMNIST",
    save_strategy="epoch",
    evaluation_strategy="epoch",
    learning_rate=1e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.001,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    logging_dir="logs",
    logging_steps=125,
    remove_unused_columns=False,
    seed=7
)
```

TrainingArguments: 학습 매개변수 클래스

- 모델 학습에 필요한 다양한 인자들을 저장, 관리
- 하이퍼파라미터 관리
- 모델 평가
- 분산 학습
- 허깅페이스 허브 연동
- 로그 추적
- .. 등등의 기능을 제공

EWHA EURON

#4 모델 실습

#6 매크로 평균 F1 점수

```
import evaluate
import numpy as np

def compute_metrics(eval_pred):
    metric = evaluate.load("f1")
```

ViT 모델의 예측결과와 실제 정답을 비교 → Macro F1-score 계산

평가 지표 불러오는 함수

- eval_pred 매개변수: 모델의 예측결과와 실제 정답이 들어있는 데이터
- Evaluate.load("f1") : F1-score를 계산하기 위한 평가 지표를 불러옴

```
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    macro_f1 = metric.compute(
        predictions=predictions, references=labels, average="macro"
    )
    return macro_f1
```

예측결과와 정답을 분리

예측 결과를 예측 클래스로 변환: [2.1,0.5,0.3] => 클래스 0

Macro F1-score 계산

#4 모델 실습

#7 ViT 모델 학습

```
trainer = Trainer(
    model_init=lambda x: model_init(classes, class_to_idx),
    args=args,
    train_dataset=subset_train_dataset,
    eval_dataset=subset_test_dataset,
    data_collator=lambda x: collator(x, transform),
    compute_metrics=compute_metrics,
    tokenizer=image_processor,
)
trainer.train()
```

Trainer: 훈련자 클래스

ViT 모델 생성

학습 조건: 학습률, 배치 크기 등등

학습 데이터셋

평가 데이터셋

앞에서 정의한 collator() 함수 사용: 이미지 전처리, 이미지와 라벨을 미니배치로 병합

Macro-F1-Score 계산

Tokenizer: image_processor 전달,

학습 시작

EWHA EURON

#4 모델 실습

#7 ViT 모델 학습 결과

Epoch	Training Loss	Validation Loss	F1-Score
1	0.7062	0.6377	0.8905
2	0.4954	0.4781	0.9166
3	0.4078	0.4357	0.9231

손실값: 0.4079

F1: 0.9231

우수하게 학습 완료

EWHA EURON

#4 모델 실습

#8 ViT 모델 성능 평가

```
outputs = trainer.predict(subset_test_dataset)
print(outputs)

y_true = outputs.label_ids
y_pred = outputs.predictions.argmax(1)

labels = list(classes)
matrix = confusion_matrix(y_true, y_pred)
display = ConfusionMatrixDisplay(confusion_matrix=matrix, display_labels=labels)
_, ax = plt.subplots(figsize=(10, 10))
display.plot(xticks_rotation=45, ax=ax)
plt.show()
```

테스트 데이터 예측

실제 정답과 예측값 분리

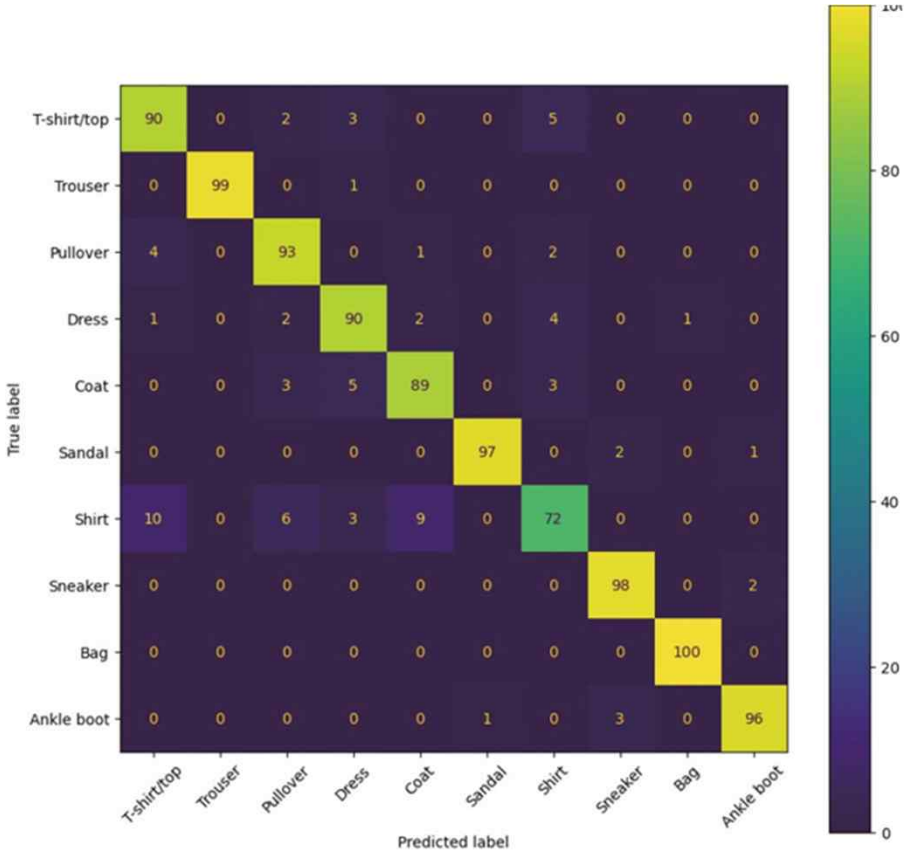
혼동 행렬 생성

- confusion_matrix(): 실제 정답고 ㅏ모델의 예측 결과를 비교하여 혼동 행렬을 생성한다.

혼동 행렬 시각화

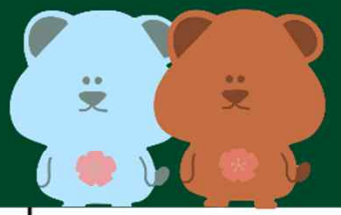
#4 모델 실습

#8 ViT 모델 성능 평가




```
'test_f1': 0.9231955290222554,
'test_runtime': 8.4356,
'test_samples_per_second': 29.636,
'test_steps_per_second': 4.283
```

92% 정확도



Swin Transformer

EWHA EURON

#1 개념

#1 스윈 트랜스포머

시프트 윈도(Shifted Window) 기술을 이용해 입력 이미지를 일정한 크기의 패치로 분할하고,
이 패치들을 쌓아 윈도를 구성.

➔ 기존 트랜스포머 모델과는 다르게 패치를 겹쳐 더 큰 효율성 제공

➔ 계층마다 어텐션이 수행되는 패치의 크기, 개수를 계층적으로 적용해 처리
: 높은 해상도와 다양한 크기를 가진 객체 효율적 처리 가능

EWHA EURON

#2 VIT와 스윈 트랜스포머 차이

#1 VIT와 스윈 트랜스포머 차이

VIT의 단점

- 획일적인 패치 크기와 위치에 대한 제약
- 패치 단위의 입력: 이미지의 공간 정보가 완전히 보존되지 않을 수 있음
- 패치수가 큰 대규모 데이터세트에는 학습에 필요한 컴퓨팅 자원이 많이 필요함

EWHA EURON

#2 VIT와 스윈 트랜스포머 차이

#1 VIT와 스윈 트랜스포머 차이

VIT의 단점

- 획일적인 패치 크기와 위치에 대한 제약
- 패치 단위의 입력: 이미지의 공간 정보가 완전히 보존되지 않을 수 있음
- 패치수가 큰 대규모 데이터세트에는 학습에 필요한 컴퓨팅 자원이 많이 필요함

-> 스윈 트랜스포머의 **로컬 윈도(Local Window)**

:입력 이미지를 처리하기 위해 사용되는 고정 크기의 작은 윈도

-> 로컬 윈도를 이동시키는 **시프트 윈도(Shifted Window)** 기술

: ViT보다 더 높은 정확도와 더 적은 매개변수로 더 넓은 범위의 이미지 크기와 종횡비를 다룸

EWHA EURON

#2 VIT와 스윈 트랜스포머 차이

#2 VIT와 스윈 트랜스포머의 네트워크 구조

- 회색 격자: 패치
- 빨간색 격자: 로컬 윈도

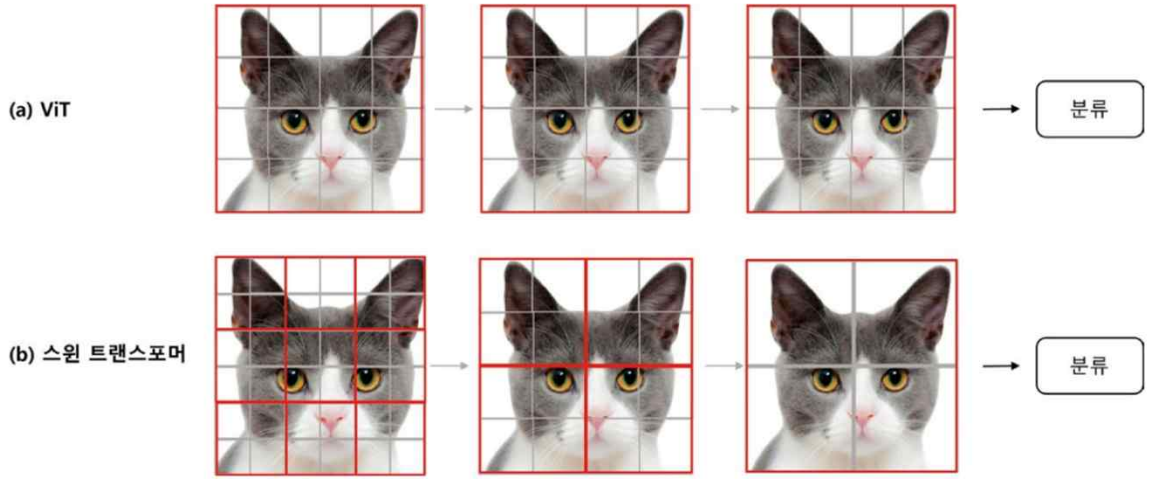


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

계층마다 전체 이미지 안에 어텐션이 수행되는 패치 크기 & 개수

=> ViT의 네트워크: 동일

=> 스윈 트랜스포머: 계층마다 다양

EWHA EURON

#2 VIT와 스윈 트랜스포머 차이

#3 VIT와 스윈 트랜스포머의 차이점

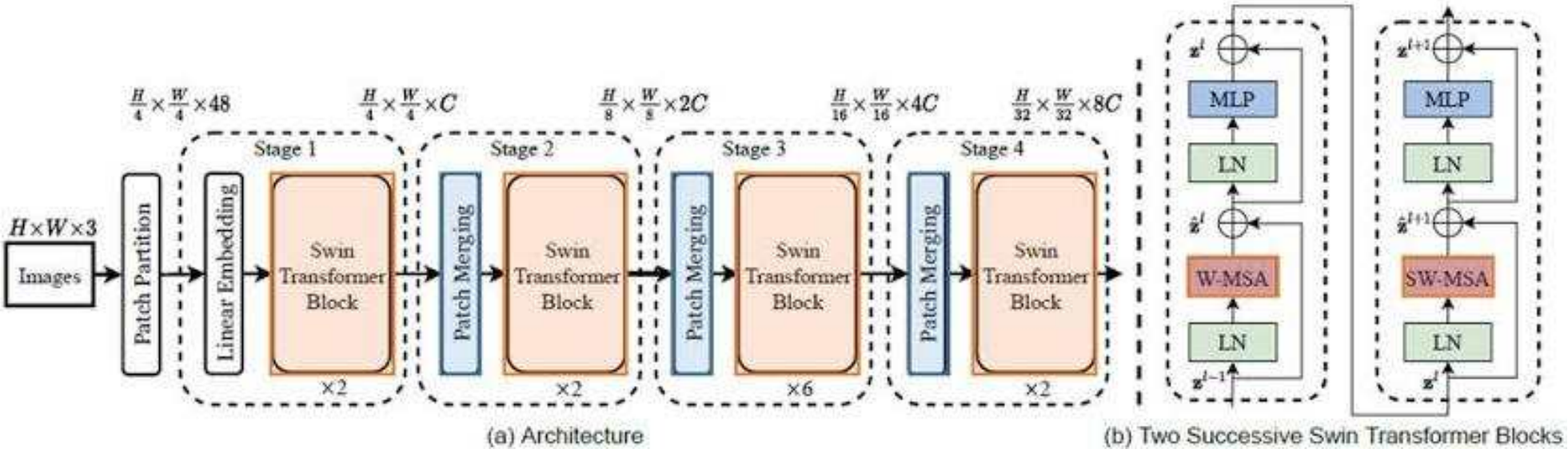
표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

EWHA EURON

#3 스윈 트랜스포머 모델 구조

#4 스윈 트랜스포머 모델 구조



네 개의 스테이지로 구성

-> 선형 임베딩, 스윈 트랜스포머 블록, 패치 병합 등의 반복 적용

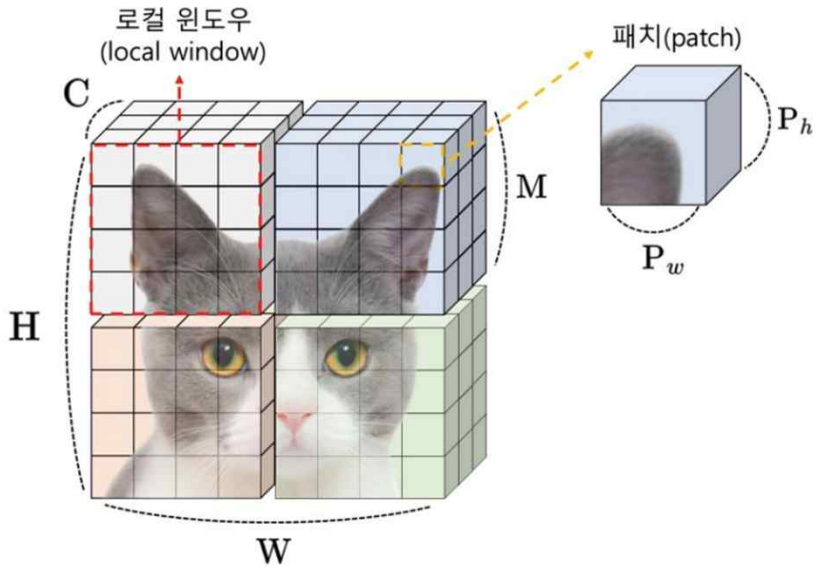
EWHA EURON

#3 스윈 트랜스포머 모델 구조

#5 패치 파티션/ 패티 병합

패치 파티션 (Patch Partition)

입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식

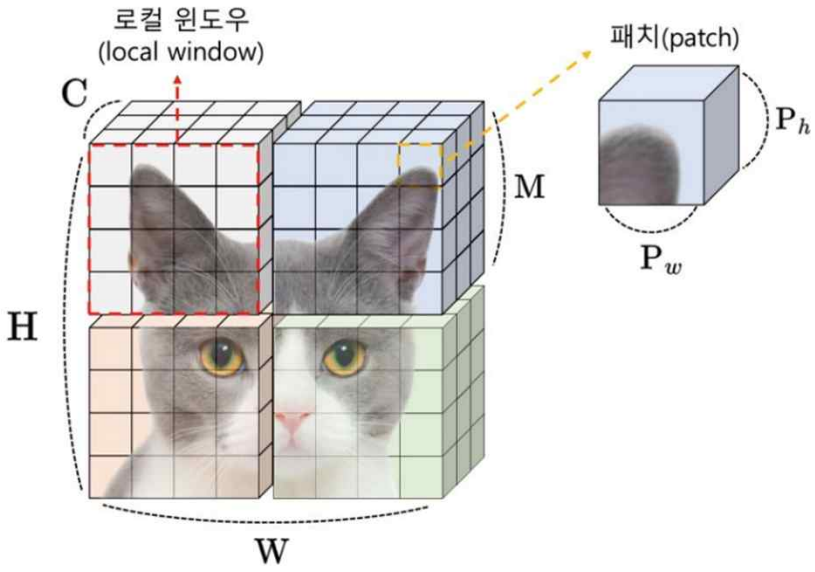


#3 스윈 트랜스포머 모델 구조

#5 패치 파티션/ 패티 병합

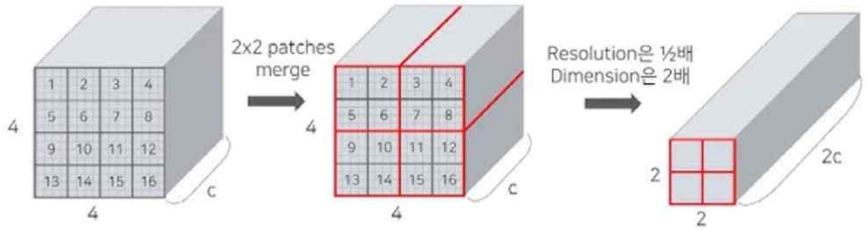
패치 파티션 (Patch Partition)

입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식



패치 병합 (Patch Merging)

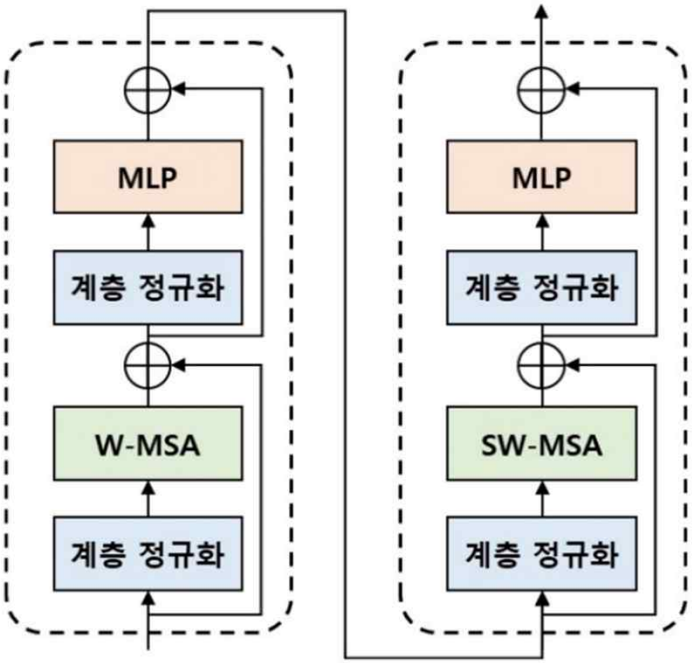
인접한 패치들의 정보를 저차원으로 축소하는 과정

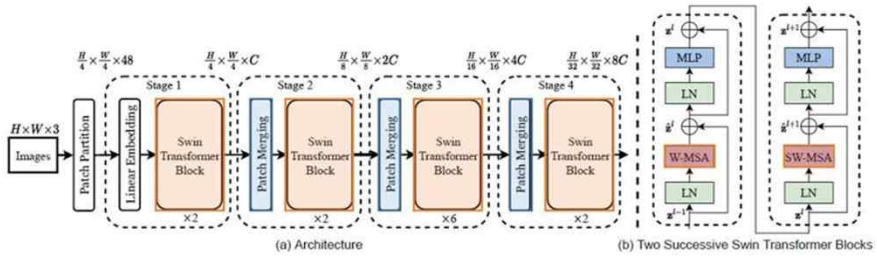


- 모델 매개변수의 수를 줄이기 위해
  이미지 패치 텐서 [C, H, W]를 [2C, H/2, W/2]로 재정렬한후,
  2C의 차원을 저차원으로 임베딩

#3 스윈 트랜스포머 모델 구조

#6 스윈 트랜스포머 블록





-> W-MSA와 SW-MSA로 구성, 순차적으로 수행됨

W-MSA

로컬 윈도를 사용하는 멀티 헤드 셀프 어텐션

- 이미지에서 일정한 크기의 윈도 영역을 설정
- 해당 영역 내 픽셀 간의 어텐션을 계산

SW-MSA

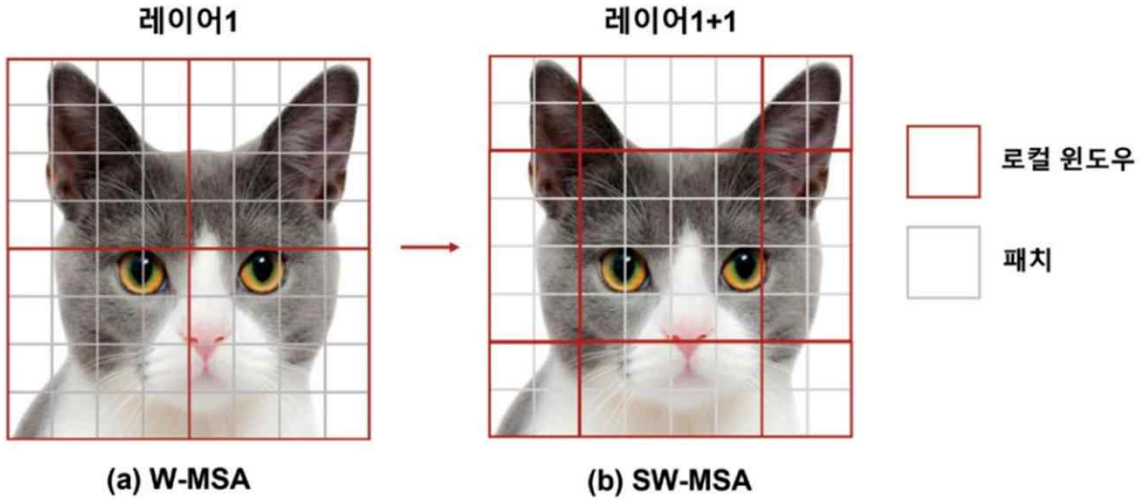
윈도를 이동한 후, 인접한 윈도 간의 셀프 어텐션을 계산

- 윈도 사이의 연결성 구축해, 윈도 간의 관계성 정보 추출

#3 스윈 트랜스포머 모델 구조

#7 W-MSA와 SW-MSA

W-MSA와 SW-MSA 어텐션 영역 비교



W-MSA 방식: 로컬 윈도(빨간색 영역) 안에서 패치 간의 셀프 어텐션 연산을 수행

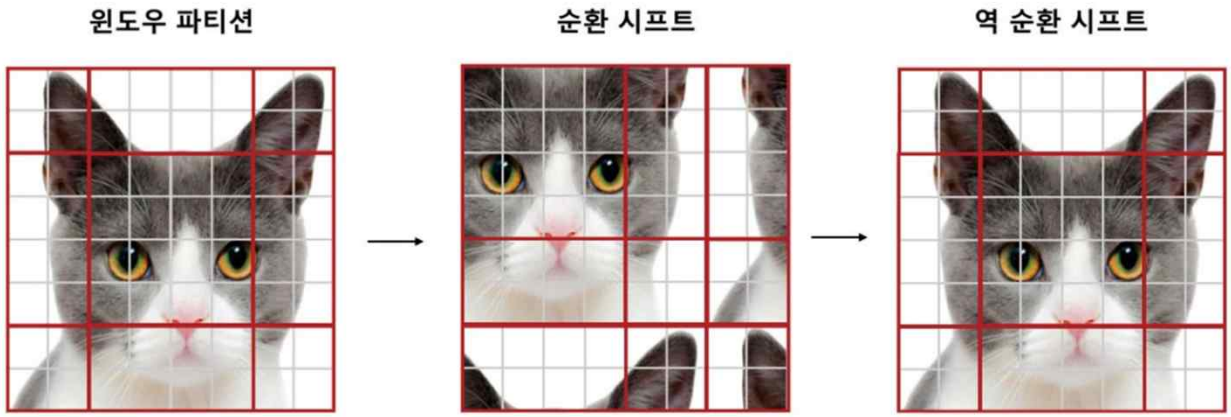
-> 효율적인 배치 계산(Effieient Batch Computation)을 통해 연산량을 대폭 감소
-> 연산속도 상승, 자연어 처리 작업에서 성능 Good!

EWHA EURON

#3 스윈 트랜스포머 모델 구조

#7 W-MSA와 SW-MSA

순환 시프트를 활용한 SW-MSA 어텐션 계산 과정



SW-MSA 방식: 로컬 윈도 간의 셀프 어텐션을 수행
- 필요한 로컬 윈도 개수 증가해 더 많은 셀프 어텐션 연산이 요구됨

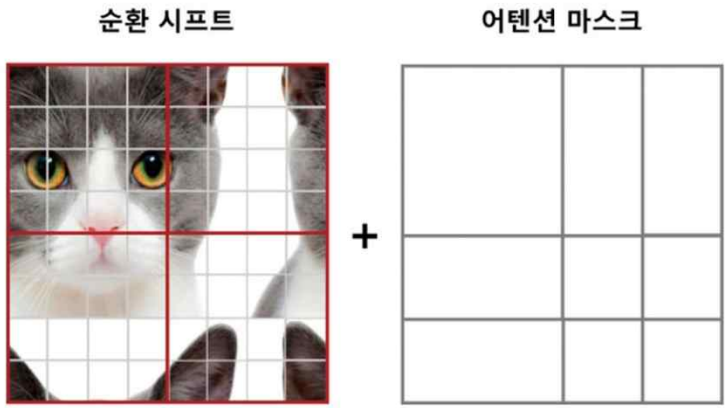
-> **순환 시프트(Cyclic Shift)**와 **어텐션마스크(Attention Mask)** 사용

EWHA EURON

#3 스윈 트랜스포머 모델 구조

#7 W-MSA와 SW-MSA

순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조



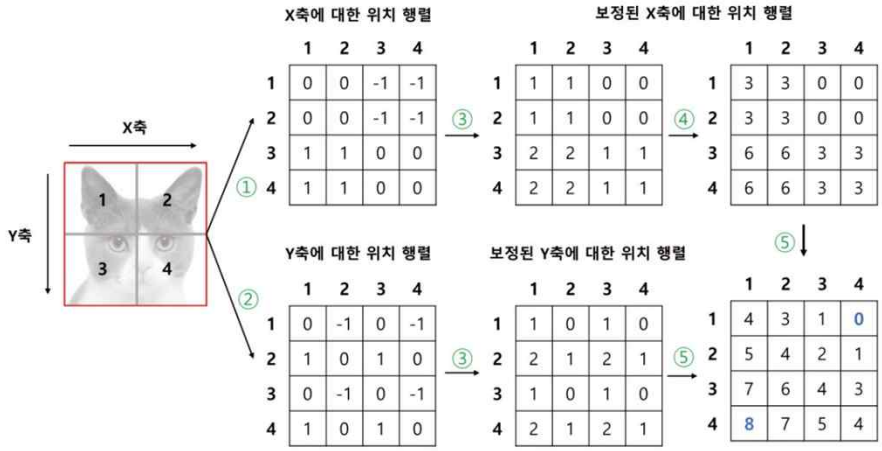
4개의 윈도 파티션에 대한 4개의 셀프 어텐션 값만으로 모든 로컬 윈도 간의 셀프 어텐션 값 계산 가능
-> 순환 시프트를 통해 이동된 윈도에 대한 정보 보존 + 불필요한 연산 줄이고 효율적인 계산 가능

EWHA EURON

#3 스윈 트랜스포머 모델 구조

#7 W-MSA와 SW-MSA

상대적 위치 편향 계산 과정



패치 간의 상대적인 거리: X축과 Y축 두 가지 방식으로 표기 가능

- X축 방식: 두 패치 간의 가로 방향 거리 의미
- Y축 방식: 세로 방향 거리 의미

#3 스윈 트랜스포머 모델 구조

#7 W-MSA와 SW-MSA

패치 간 상대 위치 계산

```
import torch

window_size = 2
coords_h = torch.arange(window_size)
coords_w = torch.arange(window_size)

coords = torch.stack(
    torch.meshgrid([coords_h, coords_w], indexing="ij")
)
coords_flatten = torch.flatten(coords, 1)

relative_coords = (
    coords_flatten[:, :, None]
    - coords_flatten[:, None, :]
)

print(relative_coords)
print(relative_coords.shape)
```

- 2 × 2 윈도에 포함된 패치 4개의 좌표를 생성
- 모든 패치 쌍의 좌표를 빼 세로/가로 방향의 상대 위치 계산
- 계산한 위치 차이 - 어텐션에 위치 정보 반영하는 데 사용

출력 결과

```
tensor([[[ 0,  0, -1, -1],
         [ 0,  0, -1, -1],
         [ 1,  1,  0,  0],
         [ 1,  1,  0,  0]],

        [[ 0, -1,  0, -1],
         [ 1,  0,  1,  0],
         [ 0, -1,  0, -1],
         [ 1,  0,  1,  0]]])
torch.Size([2, 4, 4])
```

EWHA EURON

#3 스윈 트랜스포머 모델 구조

#7 W-MSA와 SW-MSA

상대 위치 인덱스와 편향값 생성

1. 상대 위치를 인덱스로 변환

```
x_coords = relative_coords[0, :, :]
y_coords = relative_coords[1, :, :]

x_coords += window_size - 1
y_coords += window_size - 1
x_coords *= 2 * window_size - 1

print(f"X축에 대한 행렬:\n{x_coords}\n")
print(f"Y축에 대한 행렬:\n{y_coords}\n")

relative_position_index = x_coords + y_coords

print(
    f"X, Y축에 대한 위치 행렬:\n"
    f"{relative_position_index}"
)
```

- 위치 차이에 1을 더해 음수 제거
- 첫 번째 축에 3을 곱한 뒤, 두 축을 더해 0~8의 인덱스로 변환

2. 인덱스로 편향값 조회

```
num_heads = 1

relative_position_bias_table = torch.Tensor(
    torch.zeros(
        (2 * window_size - 1)
        * (2 * window_size - 1),
        num_heads
    )
)

relative_position_bias = relative_position_bias_table[
    relative_position_index.view(-1)
]

relative_position_bias = relative_position_bias.view(
    window_size * window_size,
    window_size * window_size,
    -1
)

print(relative_position_bias.shape)
```

- 인덱스를 이용해 각 패치 쌍에 대응하는 편향값 조회
- 같은 상대 위치를 가진 패치 쌍 -> 같은 편향값 사용

EWHA EURON

#4 모델 실습

#1 사전 학습된 스윈 트랜스포머 모델

실습 목표

- ImageNet-1k로 사전 학습한 Swin Transformer를 불러와 구조 확인, FashionMNIST 분류에 맞게 미세 조정

```
from transformers import SwinForImageClassification

model = SwinForImageClassification.from_pretrained(
    pretrained_model_name_or_path="microsoft/swin-tiny-patch4-window7-224",
    num_labels=len(train_dataset.classes),
    id2label={
        idx: label
        for label, idx in train_dataset.class_to_idx.items()
    },
    label2id=train_dataset.class_to_idx,
    ignore_mismatched_sizes=True
)
```

코드 설명

from_pretrained
- 사전 학습한 모델 구조와 가중치를 불러옴

num_labels
- 출력 클래스 수를 실습 데이터에 맞게 설정

ignore_mismatched_sizes=True
- 크기가 다른 분류층 가중치는 새로 초기화

EWHA EURON

#4 모델 실습

#1 사전 학습된 스윈 트랜스포머 모델

```
for main_name, main_module in model.named_children():
    print(main_name)

    for sub_name, sub_module in main_module.named_children():
        print("└", sub_name)

        for ssub_name, ssub_module in sub_module.named_children():
            print("| └", ssub_name)

            for sssub_name, sssub_module in ssub_module.named_children():
                if sssub_name == "projection":
                    print("| | └", sssub_name, sssub_module)
                else:
                    print("| | └", sssub_name)
```

- named_children()으로 하위 모듈을 순서대로 확인
- 전체 모델은 특징을 추출하는 swin과 분류하는 classifier로 구성

구성 요소 및 역할

swin.embedding
- 이미지를 패치 토큰으로 변환

swin.encoder.layers[0~3]
- 네 스테이지에서 특징 학습

swin.layernorm
- 마지막 특징 정규화

swin.pooler
- 토큰을 평균 내 이미지 대표 특징 생성

Classifier
- 클래스별 점수 계산

EWHA EURON

#4 모델 실습

#2 패치 임베딩 모듈

```
batch = next(iter(train_dataloader))
print("이미지 차원 :", batch["pixel_values"].shape)

patch_emb_output, shape = (
    model.swin.embeddings.patch_embeddings(
        batch["pixel_values"]
    )
)

print("모듈:", model.swin.embeddings.patch_embeddings)
print("패치 임베딩 차원 :", patch_emb_output.shape)
```

출력 결과

```
이미지 차원: [32, 3, 224, 224]

Conv2d(
    3, 96,
    kernel_size=(4, 4),
    stride=(4, 4)
)

패치 임베딩 차원: [32, 3136, 96]
```

- 4 × 4 합성곱을 간격 4로 적용해 패치 추출
- 224 × 224 이미지 -> 56 × 56개의 패치로 변환
- 각 패치를 96차원 특징 벡터로 표현

EWHA EURON

#4 모델 실습

#3 스윈 트랜스포머 블록

```
for main_name, main_module in (
    model.swin.encoder.layers[0].named_children()
):
    print(main_name)

    for sub_name, sub_module in main_module.named_children():
        print("└", sub_name)

        for ssub_name, ssub_module in sub_module.named_children():
            print("| └", ssub_name)
```

*스테이지 - 같은 공간 해상도로 특징을 처리하는 블록들의 묶음

- 첫 번째 스테이지: 두 개의 Swin 블록 + 패치 병합으로 구성
- 일반 윈도 / 이동 윈도를 번갈아 사용해 특징 학습
- 패치 병합으로 다음 스테이지의 입력 크기를 줄임

출력 결과

```
blocks
└ 0
| └ layernorm_before
| └ attention
| └ drop_path
| └ layernorm_after
| └ intermediate
| └ output
└ 1
| └ layernorm_before
| └ attention
| └ drop_path
| └ layernorm_after
| └ intermediate
| └ output
downsample
└ reduction
└ norm
```

- blocks[0] : 일반 윈도 기반 블록
- blocks[1] : 이동 윈도 기반 블록
- downsample : 패치 병합

EWHA EURON

#4 모델 실습

#4 SwinLayer 구조

```
print(model.swin.encoder.layers[0].blocks[0])
```

출력 결과

```
SwinLayer(
  (layernorm_before): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (attention): SwinAttention(
    (self): SwinSelfAttention(
      (query): Linear(in_features=96, out_features=96, bias=True)
      (key): Linear(in_features=96, out_features=96, bias=True)
      (value): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (output): SwinSelfOutput(
      (dense): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
  )
  (drop_path): SwinDropPath(p=0.1)
  (layernorm_after): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (intermediate): SwinIntermediate(
    (dense): Linear(in_features=96, out_features=384, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output): SwinOutput(
    (dense): Linear(in_features=384, out_features=96, bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
  )
)
```

- **SwinLayer**: 윈도 어텐션과 MLP로 구성
- **MLP**: 특징 차원을 확장한 뒤, 원래 크기로 복원함
- 어텐션과 MLP에는 각각 입력을 더하는 잔차 연결이 적용됨

출력 모듈
- layernorm_before : 어텐션 전 특징 정규화
- attention: 윈도 내부 토큰 간 관계 계산
- layernorm_after: MLP 전 특징 정규화
- intermediate: 특징 차원 확장, 96 → 384, GELU 적용
- output: 특징 차원 복원, 384 → 96
- drop_path: 학습 중 잔차 경로를 확률적으로 생략

EWHA EURON

#4 모델 실습

#5 W-MSA, SW-MSA 모듈

```
print("패치 임베딩 차원 :", patch_emb_output.shape)

W_MSA = model.swin.encoder.layers[0].blocks[0]
SW_MSA = model.swin.encoder.layers[0].blocks[1]

W_MSA_output = W_MSA(
    patch_emb_output,
    W_MSA.input_resolution
)[0]

SW_MSA_output = SW_MSA(
    W_MSA_output,
    SW_MSA.input_resolution
)[0]

print("W-MSA 결과 차원 :", W_MSA_output.shape)
print("SW-MSA 결과 차원 :", SW_MSA_output.shape)
```

출력 결과

```
패치 임베딩 차원: [32, 3136, 96]
W-MSA 결과 차원: [32, 3136, 96]
SW-MSA 결과 차원: [32, 3136, 96]
```

코드 설명

- **W-MSA:** 고정된 윈도 내부에서 어텐션 계산
- **SW-MSA:** 윈도를 이동해 이전 윈도 경계 너머의 토큰 연결
- 두 블록을 통과해도 출력 크기는 유지됨

EWHA EURON

#4 모델 실습

#6 패치 병합

```
patch_merge = model.swin.encoder.layers[0].downsample
print("patch_merge 모듈 :", patch_merge)

output = patch_merge(
    SW_MSA_output,
    patch_merge.input_resolution
)

print("patch_merge 결과 차원 :", output.shape)
```

출력 결과

```
reduction: Linear(384, 192, bias=False)
norm: LayerNorm(384)

patch_merge 결과 차원: [32, 784, 192]
```

- 인접한 2 × 2 토큰을 채널 방향으로 병합
- 토큰 수 → 1/4로 감소
- 합쳐진 특징을 선형 변환해 채널 수를 2배 증가

구분	병합 전	병합 후
공간 해상도	56 × 56	28 × 28
토큰 수	3,136	784
특징 차원	96	192

EWHA EURON

#4 모델 실습

#7 전체 차원 변화와 분류

구간	공간 해상도	토큰 수	특징 차원	Swin 블록 수
패치 임베딩	56 × 56	3,136	96	—
Stage 1 블록 출력	56 × 56	3,136	96	2
Stage 2 블록 출력	28 × 28	784	192	2
Stage 3 블록 출력	14 × 14	196	384	6
Stage 4 블록 출력	7 × 7	49	768	2
평균 풀링	—	1	768	—
분류층	—	—	클래스 수	—

- 스테이지가 깊어질수록 토큰 수는 줄고 특징 차원은 증가
- 마지막 49개 토큰을 평균 내 이미지당 768차원 벡터를 생성
- 분류층에서 클래스별 점수를 계산

EWHA EURON

#4 모델 실습

#8 스윈 트랜스포머 모델 학습

```python
from transformers import SwinForImageClassification

def model_init(classes, class_to_idx):
    model = SwinForImageClassification.from_pretrained(
        pretrained_model_name_or_path="microsoft/swin-tiny-patch4-window7-224",
        num_labels=len(classes),
        id2label={
            idx: label
            for label, idx in class_to_idx.items()
        },
        label2id=class_to_idx,
        ignore_mismatched_sizes=True
    )
    return model
```

```python
image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="microsoft/swin-tiny-patch4-window7-224"
)
```

- model_init: 데이터의 클래스 정보에 맞춰 모델 생성
- image_processor: 사전 학습 모델에 대응하는 이미지 전처리 설정 불러오기
- 기존 ViT 실습의 데이터 처리 + Trainer 학습 흐름 재사용

주요 설정

```python
args = TrainingArguments(
    output_dir="../models/Swin-FashionMNIST",
    save_strategy="epoch",
    evaluation_strategy="epoch",
    learning_rate=1e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.001,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    logging_dir="logs",
    logging_steps=125,
    remove_unused_columns=False,
    seed=7
)
```

- learning_rate=1e-5: 작은 학습률로 가중치 조정
- batch_size=16: 장치당 이미지 16개 처리
- num_train_epochs=3: 학습 데이터를 3회 반복
- weight_decay=0.001: 가중치가 과도하게 커지는 것을 억제
- save/evaluation_strategy: 매 에포크마다 저장 및 검증
- metric_for_best_model="f1”: 검증 F1이 가장 높은 모델 선택

EWHA EURON

#4 모델 실습

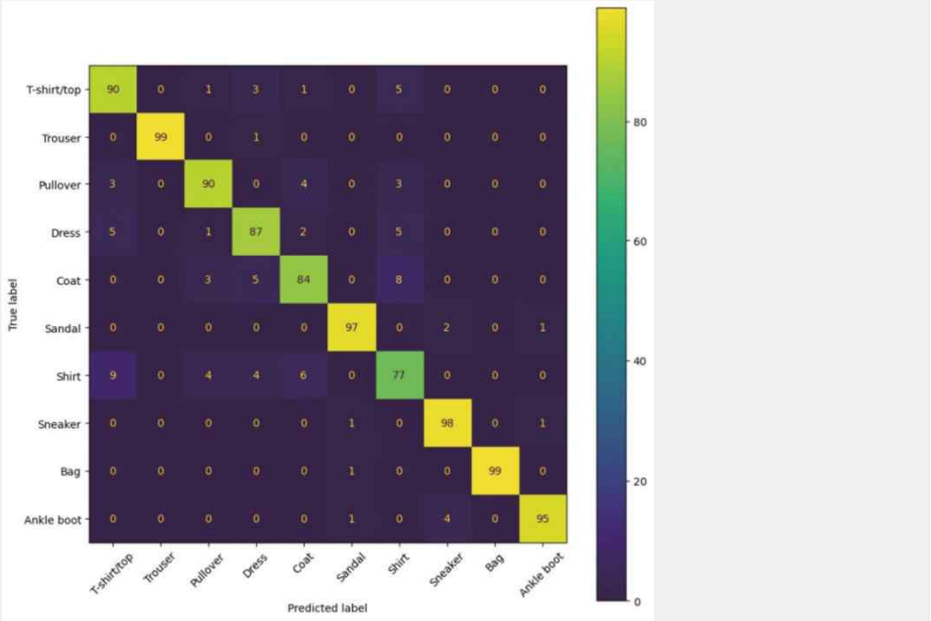
#9 스윈 트랜스포머 학습 결과

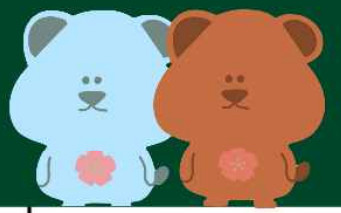
스윈 트랜스포머 학습 결과

Epoch	Training Loss	Validation Loss	F1
1	0.3720	0.3046	0.8985
2	0.2579	0.2688	0.9138
3	0.2607	0.2416	0.9159

- 스윈 트랜스포머 모델의 분류 정확도: 91.6%
- ViT 학습 결과와 비교했을 때 F1 성능 지표는 거의 동일하지만, 검증 손실 값이 대폭 낮아짐

```
# 생략
{
    'test_loss': 0.24168206751346588,
    'test_f1': 0.9159858553952347,
    'test_runtime': 4.3068,
    'test_samples_per_second': 58.048,
    'test_steps_per_second': 3.657
}
# 생략
```





CvT

Convolutional Vision Transformer

EWHA EURON

CvT

CvT는 합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델

ViT의 셀프 어텐션은 이미지 정보를 바탕으로 각 패치 간의 관계를 학습하지만, 모든 패치에 대해 동일한 크기와 간격을 사용하므로 이미지의 물체 크기나 해상도를 계층적으로 학습하기가 어려움

➔ CvT는 합성곱 신경망에서 사용하는 계층적 구조를 ViT에 적용해 이미지의 지역 특징과 전역 특징을 모두 활용해 비교적 적은 수의 매개변수로 높은 성능을 보임

합성곱 계층의 지역 특징을 추출하고 ViT의 셀프 어텐션 계층으로 전역 특징을 결합해 이미지를 처리

➔ 스윈 트랜스포머보다 더 적은 수의 매개변수로도 비슷한 수준의 성능을 보임

EWHA EURON

CvT 모델의 계층적 특징 구조 예시

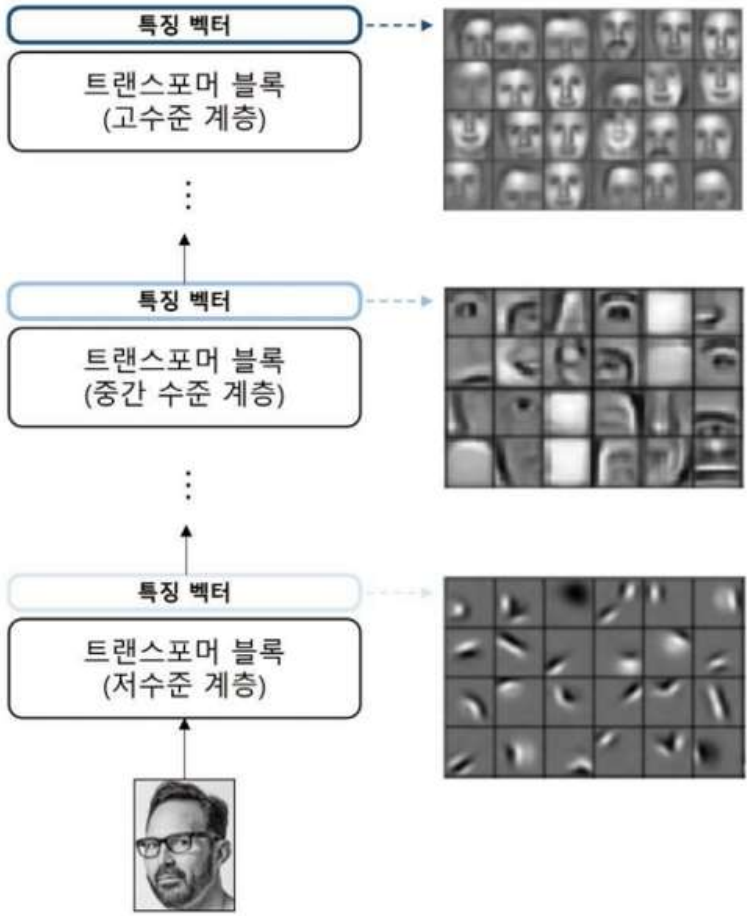


그림 10.14 CvT 모델의 계층적 특징 구조 예시

얼굴 이미지가 CvT 모델의 입력으로 사용된다면 저수준 계층에서 선 또는 점에 대한 특징을 표현하고, 중간 수준에서는 눈이나 귀와 같은 특징을 표현하며, 고수준 계층에서는 얼굴 구조에 대한 특징을 표현

➔ CvT의 계층형 구조는 물체의 크기나 해상도를 더 잘 표현한다고 할 수 있음

EWHA EURON

ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

ViT는 이미지를 패치 단위로 분할하고, 이를 트랜스포머 모델의 입력으로 사용하는 방식을 사용했다면, CvT는 이미지를 처리하기 전에 합성곱 연산을 통해 특징 맵을 추출함. 이를 통해 유의미한 패턴을 추출하고 일정한 크기의 패치로 나눠 트랜스포머 모델의 입력으로 사용하고 입력되는 패치들은 겹쳐 학습

➔ ViT는 패치 임베딩과 선형 임베딩을 사용하지만, CvT 모델은 합성곱 연산을 통해 이미지를 학습

CvT 모델은 토큰에 대한 위치 임베딩을 적용하지 않아도 모델의 성능 저하 없이 다양한 해상도를 가진 이미지를 처리할 수 있게 됨

EWHA EURON

합성곱 토큰 임베딩

합성곱 토큰 임베딩이란 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩으로, ViT에 계층적인 합성곱 신경망의 성질을 추가하기 위해 사용

기존 트랜스포머 모델은 임베딩을 사용해 셀프 어텐션을 계산하고 출력을 생성하므로 이미지의 근본적인 2D 모양의 구조를 잃게 됨

반면에 CvT 모델은 이미지의 2D 구조를 보존하면서도 셀프 어텐션을 계산하는 방식을 사용

이미지 패치를 2D 합성곱 임베딩을 사용해 쿼리, 키, 값 임베딩으로 변환하고 이렇게 변환된 임베딩을 사용해 셀프 어텐션을 계산하고 출력을 생성

EWHA EURON

2D 합성곱 기반의 어텐션 임베딩

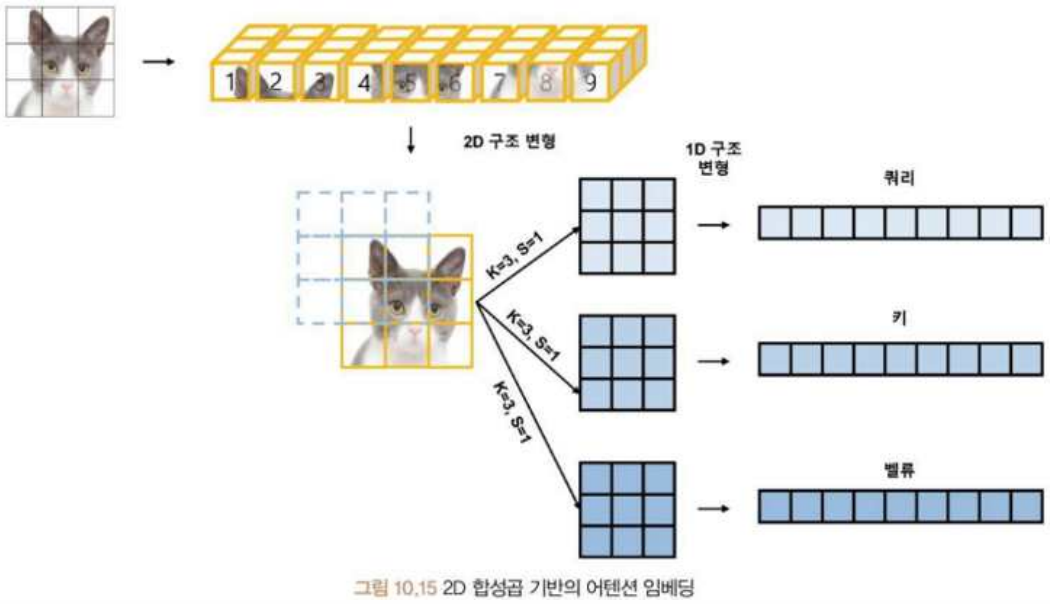


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

이미지에 3 × 3 커널 크기(K), 1 간격(S)의 커널을 적용하면 패치 크기는 3 × 3으로 생성됨

각각의 패치를 쿼리, 키, 값에 대해 합성곱 연산을 적용하고 1차원 구조로 재배열해 기존 셀프 어텐션 방법으로 학습하는 방식으로 진행

➔지역 특징을 학습할 수 있고, 시퀀스 길이를 줄이는 동시에 토큰 특징의 차원을 증가시킴.
다운 샘플링과 특징 맵의 수를 늘리게 됨

EWHA EURON

어텐션에 대한 합성곱 임베딩

어텐션에 대한 합성곱 임베딩이란 2D로 재구성된 토큰 맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산을 의미

CvT 모델은 매개변수의 수를 줄이기 위해 쿼리, 키, 값의 차원을 축소

EWHA EURON

2D 합성곱 기반의 축소된 어텐션 임베딩

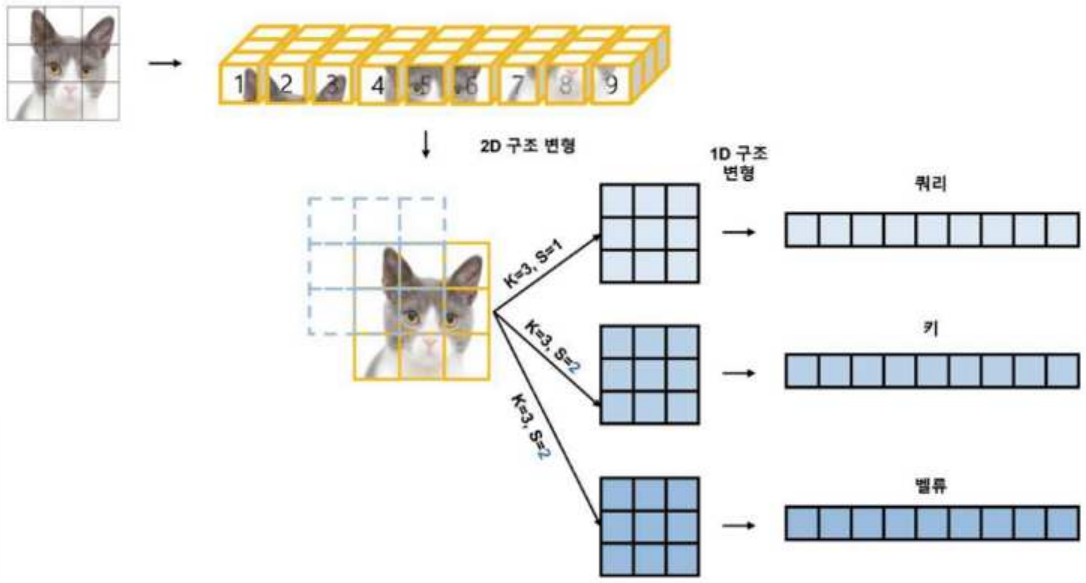
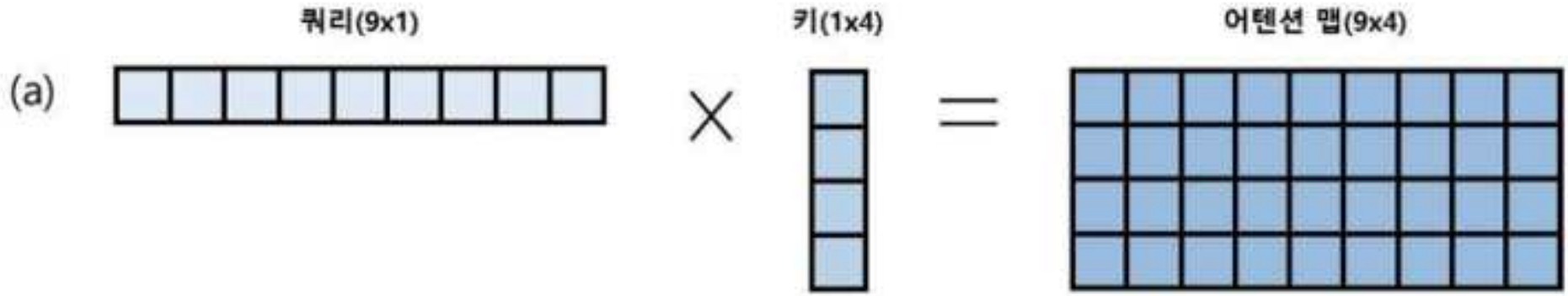


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

3 × 3 커널 크기(K), 1 간격(S)의 커널을 적용해 9개의 이미지 패치를 생성하며, 키와 값은 3 × 3 커널 크기(K), 2 간격(S)으로 커널을 적용해 4개의 이미지 패치를 생성

어텐션 맵



9개의 쿼리 패치 임베딩과 4개의 키 패치 임베딩에 대한 중요도를 의미하는 어텐션 맵을 생성

EWHA EURON

셀프 어텐션 벡터

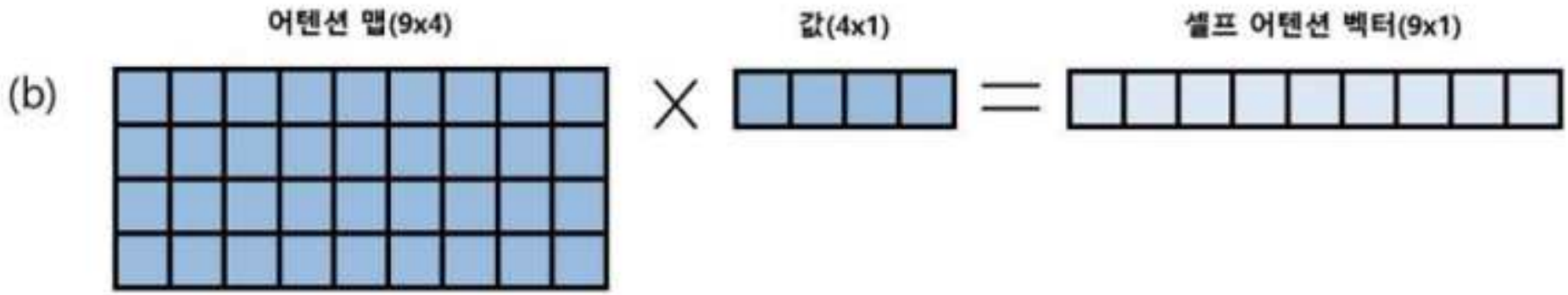


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

어텐션 맵이 생성되면 위와 같이 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 가중치를 반영

EWHA EURON

이미지 패치 개수

셀프 어텐션 벡터를 확인해 보면 입력 벡터와 출력 벡터의 패치 길이가 동일한 것을 확인할 수 있음

➔ 계산해야 하는 이미지 패치 개수가 줄어들어 연산량이 감소

CvT 모델은 트랜스포머 계층이 깊어질수록 패치 간의 관계를 학습하는 길이를 축소시키는 대신에 벡터의 차원을 증가시켜 모델의 매개변수 수를 대폭 감소시킴

➔ 다시 말해 이미지 패치 크기가 줄어든 만큼 임베딩 차원을 늘리는 방법을 택해 네트워크의 안정성을 향상시키고 모델 설계를 단순화

EWHA EURON

모델 실습

앞선 ViT 모델의 실습 코드를 활용해 CvT 모델을 미세 조정

- CvT는 허깅 페이스 라이브러리에서 제공하는 사전 학습된 microsoft/cvt-21 모델을 사용
- 학습에 사용되는 데이터세트는 앞선 ViT 예제의 코드를 활용

아래는 CvT 모델 적용 시 주요한 변경 점을 보여줌

```
from transformers import AutoImageProcessor

...

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="microsoft/cvt-21"
)
```

EWHA EURON

이미지 크기

```
transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize(
            size=(
                image_processor.size["shortest_edge"],
                image_processor.size["shortest_edge"]
            )
        ),
        transforms.Lambda(lambda x: torch.cat([x, x, x], 0)),
```

CvT 모델은 이미지 크기를 조절할 때 shortest_edge 키를 사용해 전처리를 진행

shortest_edge란 이미지의 너비나 높이 중 더 작은 값을 의미

EWHA EURON

shortest_edge

```
        transforms.Lambda(lambda x: torch.cat([x, x, x], 0)),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)

...
```

microsoft/cvt-21 모델은 모든 입력 이미지의 크기를 가장 짧은 이미지 길이로 정규화해 사용하므로 높이나 너비에 대한 키 대신 shortest_edge를 사용

EWHA EURON

예제 10.21 사전 학습된 CvT 모델

CvT 모델에 맞는 전처리 과정을 적용했다면 모델의 구조를 출력

```
from transformers import CvtForImageClassification

model = CvtForImageClassification.from_pretrained(
    pretrained_model_name_or_path="microsoft/cvt-21",
    num_labels=len(train_dataset.classes),
    id2label={idx: label for label, idx in train_dataset.class_to_idx.items()},
    label2id=train_dataset.class_to_idx,
    ignore_mismatched_sizes=True
)
```

EWHA EURON

모델 구조

```
for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("└", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("    └", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("      └", sssub_name)
```

EWHA EURON

출력 결과

```
cvt
└ encoder
   └ stages
     └ 0
     └ 1
     └ 2
layernorm
classifier
```

CvT 모델은 크게 세 개의 스테이지로 구성된 cvt와 계층 정규화, 분류기로 구성

EWHA EURON

분류기

```
cvt
└ encoder
   └ stages
     └ 0
     └ 1
     └ 2
layernorm
classifier
```

앞선 스윈 트랜스포머와의 차이점은 분류기 이전에 풀링 계층이 존재하지 않는다는 것

CvT 모델은 이미 합성곱 계층을 활용해 지역 특징 정보와 전역 정보를 파악하므로 풀링을 적용하지 않음

EWHA EURON

예제 10.22 CvT 모델의 스테이지 구조

스테이지별 이미지 크기와 임베딩 차원을 살펴보면 이미지 사이즈는 스테이지별로 4배, 8배, 16배 감소되며, 반비례하게 임베딩 차원이 커짐

아래는 첫 번째 스테이지 구조를 출력

```
stages = model.cvt.encoder.stages
print(stages[0])
```

EWHA EURON

CvtEmbeddings

```
CvtStage(
  (embedding): CvtEmbeddings(
    (convolution_embeddings): CvtConvEmbeddings(
      (projection): Conv2d(3, 64, kernel_size=(7, 7), stride=(4, 4), padding=(2, 2))
      (normalization): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
  )
```

각 스테이지는 크게 CvtEmbeddings와 CvtLayer로 구성

CvtEmbeddings는 입력 이미지에 대한 2D 합성곱 연산과 계층 정규화로 이뤄져 있으며 이미지를 패치 단위로 잘라 임베딩하는 역할

EWHA EURON

첫 번째 스테이지

```
(convolution_embeddings): CvtConvEmbeddings(
  (projection): Conv2d(3, 64, kernel_size=(7, 7), stride=(4, 4), padding=(2, 2))
  (normalization): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
)
```

입력 이미지는 첫 번째 스테이지의 CvtEmbeddings에 적용되는데, 7 x 7 커널 크기, 4 간격, 2 패딩 크기로 합성곱 연산을 적용

224 x 224 크기의 이미지에 합성곱 연산을 적용한다면 (224−7+2 x 2)/4+1=56.25의 크기가 반환되고, 소수점이 발생하므로 버림 처리하면 56 x 56 크기의 텐서가 생성됨

➔ 출력 채널의 크기가 64이므로 [N, 64, 56, 56]의 텐서가 반환됨

EWHA EURON

CvtLayer

```
  (layers): Sequential(
    (0): CvtLayer(
      (attention): CvtAttention(
        (attention): CvtSelfAttention(
          (convolution_projection_query): CvtSelfAttentionProjection(
            (convolution_projection): CvtSelfAttentionConvProjection(
                  (convolution): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=64, bias=False)
                    (normalization): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
            (linear_projection): CvtSelfAttentionLinearProjection()
          )
```

이러한 CvtLayer에서 3,136(56 × 56)개의 이미지 패치에 대한 셀프 어텐션이 수행됨

EWHA EURON

키와 값 임베딩

```
          (convolution_projection_key): CvtSelfAttentionProjection(
            (convolution_projection): CvtSelfAttentionConvProjection(
                  (convolution): Conv2d(64, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
groups=64, bias=False)
                    (normalization): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
            (linear_projection): CvtSelfAttentionLinearProjection()
          )
          (convolution_projection_value): CvtSelfAttentionProjection(
            (convolution_projection): CvtSelfAttentionConvProjection(
                  (convolution): Conv2d(64, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
groups=64, bias=False)
                    (normalization): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
```

매개변수가 축소된 어텐션 임베딩을 계산하기 위해 키와 값 임베딩의 간격을 1보다 큰 2로 설정해 차원을 축소

EWHA EURON

예제 10.23 셀프 어텐션 적용

```
batch = next(iter(train_dataloader))
print("이미지 차원 :", batch["pixel_values"].shape)

patch_emb_output = stages[0].embedding(batch["pixel_values"])
print("패치 임베딩 차원 :", patch_emb_output.shape)
```

셀프 어텐션 모듈에 이미지를 전달했을 때 셀프 어텐션의 결괏값을 보여줌

EWHA EURON

패치 임베딩

```
batch_size, num_channels, height, width = patch_emb_output.shape
hidden_state = patch_emb_output.view(batch_size, num_channels, height * width).permute(0, 2, 1)
print("셀프 어텐션 입력 차원 :", hidden_state.shape)
```

합성곱 계층을 통해 계산된 패치 임베딩은 가로와 세로 크기로 재정렬해 CvtLayer에 입력됨

```
attention_output = stages[0].layers[0].attention.attention(hidden_state, height, width)
print("셀프 어텐션 출력 차원 :", attention_output.shape)
```

CvtLayer의 attention.attention 계층을 통해 셀프 어텐션이 수행됨

EWHA EURON

셀프 어텐션 출력 차원

```
이미지 차원 : torch.Size([32, 3, 224, 224])
패치 임베딩 차원 : torch.Size([32, 64, 56, 56])
셀프 어텐션 입력 차원 : torch.Size([32, 3136, 64])
셀프 어텐션 출력 차원 : torch.Size([32, 3136, 64])
```

모듈의 입력 차원과 출력 차원의 형태가 동일

EWHA EURON

예제 10.24 CvT 모델 학습

이제 FashionMNIST 데이터세트를 활용해 CvT 모델을 미세 조정

아래는 예제 10.8 ViT 모델 학습 코드를 일부 변경한 것

```
from transformers import CvtForImageClassification

...

def model_init(classes, class_to_idx):
    model = CvtForImageClassification.from_pretrained(
        pretrained_model_name_or_path="microsoft/cvt-21",
        num_labels=len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
        ignore_mismatched_sizes=True
    )
    return model

...
```

전처리 과정

```
image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="microsoft/cvt-21"
)

transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize(
            size=(
                image_processor.size["shortest_edge"],
                image_processor.size["shortest_edge"]
            )
        ),
        transforms.Lambda(lambda x: torch.cat([x, x, x], 0)),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)

...
```

변경된 곳은 사전 학습된 모델, 전처리 과정, 출력 디렉터리

EWHA EURON

TrainingArguments

```
args = TrainingArguments(
    output_dir="../models/CvT-FashionMNIST",
    save_strategy="epoch",
    evaluation_strategy="epoch",
    learning_rate=1e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.001,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    logging_dir="logs",
    logging_steps=125,
    remove_unused_columns=False,
    seed=7
)

...
```

ViT 모델 미세 조정 방식과 동일하게 훈련자 클래스를 통해 CvT 모델의 학습을 수행

EWHA EURON

CvT 학습 결과

학습이 완료됐다면 CvT-FashionMNIST 폴더에 체크포인트마다 학습 결과, 성능 지표, 하이퍼파라미터 등이 저장됨

아래는 CvT 모델의 학습 결과를 정리한 것

Epoch	Training Loss	Validation Loss	F1
1	0.7737	0.3269	0.8914
2	0.7008	0.2596	0.9172
3	0.6438	0.2460	0.9173

EWHA EURON

분류 정확도

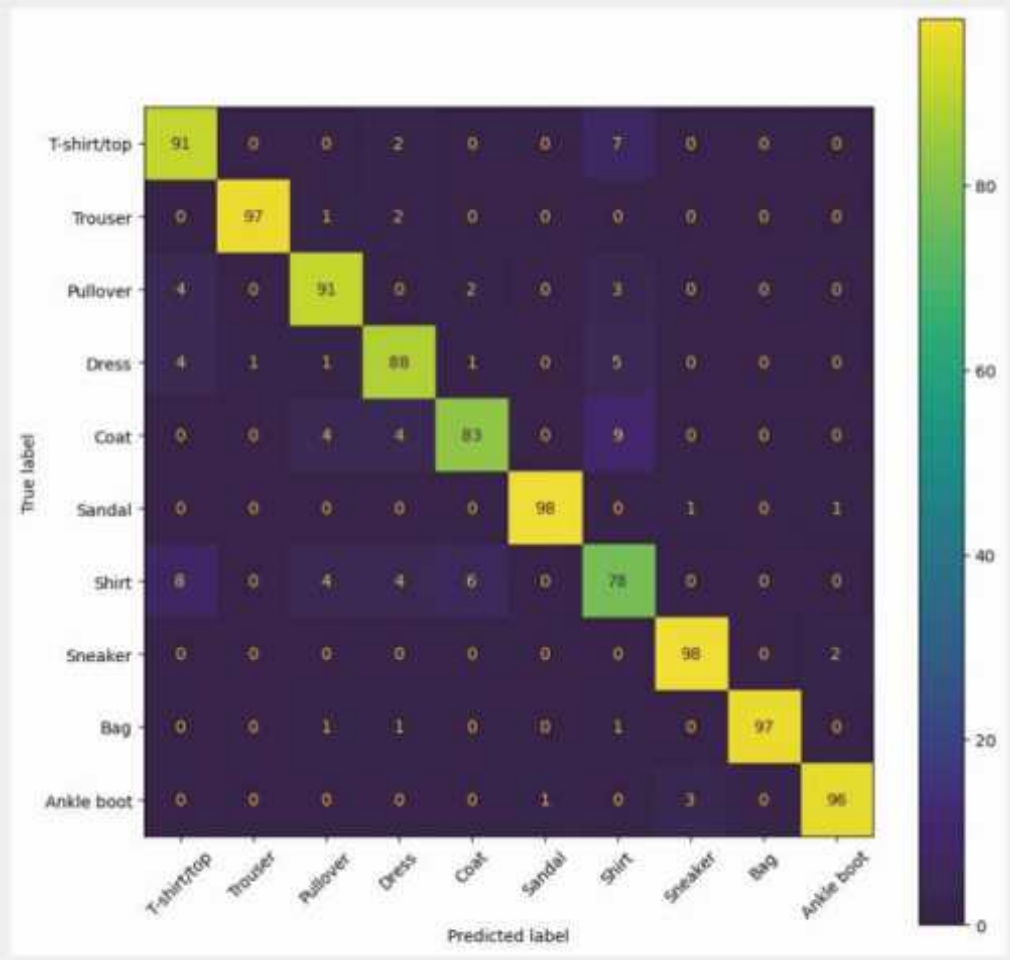
```
# 생략
{
    'test_loss': 0.246015578508377708,
    'test_f1': 0.917351060925814,
    'test_runtime': 5.583,
    'test_samples_per_second': 44.778,
    'test_steps_per_second': 2.821
}
# 생략
```

테스트 데이터세트로 추론했을 때 CvT 모델은 91.3%의 분류 정확도를 가짐

스윈 트랜스포머 모델과 CvT 모델의 F1 점수 결과를 비교했을 때 0.9159에서 0.9173으로 소폭 개선

EWHA EURON

혼동 행렬



다음 출력 결과는 앞선 ViT 모델 성능 평가를 활용해 혼동 행렬로 CvT 모델의 성능을 평가한 것

CvT 모델과 ViT 모델을 비교하면 셔츠 카테고리에 대한 성능이 개선됐으며, 스윈 트랜스포머와 비슷한 성능을 보이는 것을 확인할 수 있음

EWHA EURON

비전 트랜스포머 모델 비교

	ViT	스윈 트랜스포머	CvT
모델 크기	327.325 MB	105.227 MB	120.791 MB
F1-점수	0.9231	0.9159	0.9173
초당 처리 수	29.636 eval/s	58.048 eval/s	44.778 eval/s

10장에서 소개한 3개의 모델 크기 및 성능을 종합하면 위의 표와 같음

FashionMNIST 데이터세트에서 비전 트랜스포머 계열 모델은 작은 크기의 학습 데이터세트에서도 높은 정확도와 우수한 성능을 보임

EWHA EURON

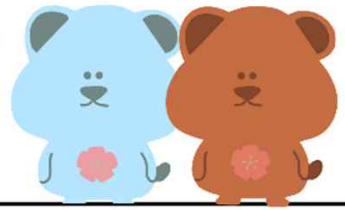
비전 트랜스포머 모델

가장 일반화 성능이 좋았던 모델은 CvT 모델, 모델 크기 대비 가장 우수한 성능은 스윈 트랜스포머 모델, F1-점수가 가장 높은 모델은 ViT 모델

➔ 비전 트랜스포머 모델은 분류 문제 해결에 있어 매우 유용한 모델임을 확인할 수 있음

EWHA EURON

THANK YOU



EWHA EURON



9주차 질문

9장: ViT, Swin Transformer, CvT

EWHA EURON

질문1: 이은수

Q. ViT는 귀납적 편향이 거의 없어 대용량 데이터가 있어야 성능이 좋다고 배웠습니다. 그러나 실습(예제 10.8)에서는 FashionMNIST 클래스 당 1000장만 사용하였으며, 28 x 28 이미지를 224 x 224로 늘리고 같은 채널을 3번 복사한 데이터였는데도 3 epoch만에 학습이 잘 되었습니다. 이는모델이 이미 ImageNet-21k로 사전학습 되어 있기 때문으로 보입니다. 그렇다면 사전 학습이 귀납적 편향을 대신한다고 볼 수 있는지 같이 논의해보고 싶습니다.

EWHA EURON

질문2: 이효주

Q. Swin Transformer는 패치 병합을 하면서 공간 크기는 줄이고 채널 수는 늘리는 방식으로 특징을 학습한다고 이해했습니다. 그런데 이렇게 인접한 패치들을 계속 합치다 보면 작은 객체나 경계처럼 세세한 정보는 줄어들 수도 있지 않을까 하는 생각이 들었습니다. 교재에서는 이미지 분류 실습을 중심으로 다루고 있는데, 작은 객체를 찾거나 위치를 세밀하게 구분해야 하는 문제에서도 패치 병합이 똑같이 유리할까요? 또 패치 병합으로 얻는 장점과 세부 정보가 줄어들 가능성 사이의 균형은 보통 어떻게 맞추는지도 궁금합니다.

EWHA EURON

논의: 조수진

Q. Swin Transformer는 연산량을 줄이기 위해 Window 단위로 Self-Attention을 수행하고, Shifted Window를 통해 서로 다른 Window 사이의 정보를 교환합니다. 반면 ViT는 Global Self-Attention을 사용하기 때문에 모든 패치가 직접 정보를 교환할 수 있습니다.
이러한 구조적 차이를 고려했을 때, 이미지에서 멀리 떨어진 영역 사이의 관계를 파악하는 것이 중요한 경우 Swin Transformer가 ViT보다 불리하게 작용할 가능성은 없을까요? 또한 여러 층에 걸친 정보 교환만으로도 전역적인 관계를 충분히 학습할 수 있다고 볼 수 있을지 그리고 이러한 간접적인 정보 전달 방식이 실제 이미지 인식 성능에 어떤 영향을 미칠지 논의해 보고 싶습니다.

EWHA EURON